

Exercise Session 1

Welcome to the first exercise session of the Computer Systems Performance course!

The session is divided into two parts.

The first part consists of forming a group of up to 3 members and generating an ssh-key. The second part of this session is about getting familiar with using the terminal with bash. Linux & Mac users can get started using their terminal, as it already uses bash. Windows users require other means to work locally with these tools. We recommend the following solution: Install the [Ubuntu](#) app from Microsoft Store. If you have an older (pre-Windows 10 version of Windows, reach out to us, we can suggest a different solution for you.)

Part 1

In this part you will form a group, generate an SSH key if you don't already have one, and send the public part of that key to us (dayo@itu.dk and pito@itu.dk).

This will allow us to grant you access to the server which can be used for the projects in this course.

Step 1 – Forming a group

For the projects in this course, you will need a group of up to 3 members.

If you haven't already found your members, this is the part where you mingle and find some. If you have difficulty with this, reach out to us.

It is also OK to do the projects individually, but we recommend that you work in groups, it helps.

Enter your group selection at LearnIT under "Form your project groups". You are welcome to give creative names to your groups, but, regardless, please give a unique number to your group. If the number of your group is 0, your username on the server will be *group0*.

Step 2 – Generating an SSH-key

NOTE: If you already have an ssh-key pair skip this step.

1. Open Terminal
2. Execute the command:

```
$ ssh-keygen -t rsa
```
3. Accept the default location to keep the key
4. You can decide whether you want a passphrase or not (it is recommended that you have one though)
5. Following these steps, you should have a private and public ssh-key file in `~/.ssh`

- a. The private key file that you must never share is: `~/.ssh/id_rsa`
- b. The **public key** file that you can share freely is: `~/.ssh/id_rsa.pub`

Step 3 – Send your public key to us

Now that you have created an SSH-key pair, you should send it to us (dayo@itu.dk and pito@itu.dk). Please put your group number to the subject line of the email and send all the SSH keys for your group altogether in one email.

Part 2

In this part you should get accustomed to navigating in the terminal and using command line tools such as screen, tmux, vim, emacs, and crontab.

Logging into the class server

We would be adding your public keys as authorized keys in your group accounts, after that you can access the server that we have set up for the class.

Linux & Mac & Windows:

1. Open terminal, make sure your private key is under `~/.ssh` folder
2. `$ ssh group<yourgroup#>@dionysos.itu.dk`

Server access outside ITU

If you want to access the server while not connected to the ITU++ network, you need to set up a VPN. This is quite simple, and is explained here: <https://itustudent.itu.dk/campus-life/it-services/remote-access>

Linux users can just follow the Windows guide, as FortiClient is available both as .deb and .rpm.

Navigation, creation of directories and files

1. Change directory using `cd` to your home directory(`~`) or another destination you see fit for storing files used for this exercise.
 - a. `cd ~`
2. Use the command `mkdir` to create a new directory with a name of your choosing
 - a. `mkdir <name>`
3. Change to the directory you just created and create a file using `touch`
 - a. `touch <filename>`

VSCode SSH extension

This extension is super handy to edit files on the server, it is optional if you want to use it, but it is probably easier to use than VIM or EMACS.

1. Install the “*Remote – SSH*” extension for vscode

2. Click ctrl+shift+p and search for "Remote-SSH: Connex to Host"
3. The first time you connect, click "Add New SSH Host" and enter `ssh group<groupnumber>@dionysos.itu.dk`

VIM

Beginner's Guide: <https://www.linux.com/learn/vim-101-beginners-guide-vim>

Cheat Sheet: <https://vim.rtorr.com/>

1. Add the following text "I am a student at ITU" inside the file you created using vim
 - a. vim <filename>
 - b. Access insert mode using "i" and write the text
2. Now copy and paste that sentence 10 times
 - a. Exit insert mode with Esc
 - b. Move the cursor to the beginning of the line and press "yy" (this is known as yanking the line)
 - c. Start pasting using "p". Alternatively use "10p" to paste 10 times instantly.
3. Save and exit the file
 - a. Save the file with ":w"
 - b. Exit the file with ":q"
 - c. Alternatively if you want to save and exit in one step chain the two commands together like this ":wq".

GNU EMACS

Guide: <https://www.gnu.org/software/emacs/manual/>

Screen, Tmux, and Crontab

When working on servers it is often the case that you need to run processes/experiments that take a long time to finish, perhaps several days.

In this case keeping a remote connection to the server is not sustainable and even if it was most servers automatically closes the connection if no interaction has occurred within a time limit. To solve this we can use tools such as screen and tmux. These allow us to create a session that we can leave running on the server.

What if we need to run an experiment according to a schedule or run something on a(n) hourly/daily/weekly/monthly basis. For that we can use crontab.

Use the links below to familiarize yourself with screen and crontab aside from this short demo.

1. Create a new session
 - a. `tmux new-session -s my-session`
2. Create a new file that will serve as your bash script
 - a. `touch filename`
3. Use vim to enter and edit the script. Add the following line to the script

- a. `echo "Some text" >> filename`
4. Try to make a crontab that appends a new line to a file every minute. Use the file just created
5. Now press `Ctrl-b + c` to create a new window
6. Edit your initial shell script using vim to append to another file and then detach from your session
 - a. `Ctrl-b + d`
7. Reattach to the session
 - a. `tmux attach -t my-session`
8. Exit the vim editor, detach from your current session and kill your session
 - a. `:wq` - to save and exit vim
 - b. `Ctrl-b + d` - to detach from current session
 - c. `tmux kill-session -a -t my-session` - to kill your session
9. List your current sessions. You should see no active sessions
 - a. `tmux ls`

Screen guide: <https://linuxize.com/post/how-to-use-linux-screen/>

Tmux guide: <https://tmuxcheatsheet.com/>

Crontab: <https://www.computerhope.com/unix/ucrontab.htm>

Crontab Help: <https://crontab.guru/>

Need a primer for command line environment, check out the following [tutorial](#). In general, <https://missing.csail.mit.edu/> has tutorials on lots of useful tools.

That's all folks! Don't hesitate to ask us questions!